Deep Learning-based Search Space Reconstruction Empowers Evolution Algorithm for Optimization

Yaotong Song, Kaiyu Wang, Zhi-Hui Zhan, *Fellow, IEEE*,
Zhenyu Lei, and Shangce Gao, *Senior Member, IEEE*

***Abstract*—Evolutionary algorithms serve as a pivotal tool in addressing black-box problems, finding widespread applications across diverse academic disciplines and engineering domains. Despite their utility, these algorithms often confront challenges when navigating high-dimensional search spaces, impeding a comprehensive exploration of potential solutions. Relying solely on the algorithm's exploration capabilities proves insufficient for fully leveraging the information inherent in high-dimensional search spaces. To unlock the full potential of high-dimensional solution spaces, we introduce a deep learning method for the reconstruction of the search space. Specifically, discrete data sampled by evolutionary operators during the exploration process are collected, and a uniquely designed fully connected neural network is employed to reconstruct the exploration paths. The neural network's robust fitting capability facilitates the transformation of initially discrete sampled information into a continuous form. By capitalizing on the reconstructed solution space information, the algorithm excels in identifying superior solutions. We refer to this method of deep learning-based search space reconstruction evolution strategy algorithm (DLES). The effectiveness of DLES is validated across multiple datasets, including CEC 2014, CEC 2018, CEC 2022 and BBOB. Experimental results, compared to several state-of-the-art algorithms, affirm the superiority of the DLES algorithm.**

***Index Terms*—Evolutionary computation, deep learning, search space reconstruction, artificial neural network, single-objective optimization.**

I. Introduction

EVOLUTIONARY computation (EC) is a powerful tool to address complex optimization problems [1]. Inspired by biological evolution, evolutionary algorithms assess the fitness of real-valued solutions to promote the progress of search. The best solution vectors are retained while other solutions that are not favorable to evolution are discarded. The surviving high-quality solutions are subsequently subjected to crossover and mutation to generate the next generation. This crossover approach addresses the limitations of traditional algorithms that have massive variables and are prone to prolonged convergence, making them difficult to train [2].

This research was partially supported by the Japan Society for the Promotion of Science (JSPS) KAKENHI under Grant JP23K24899, Japan Science and Technology Agency (JST) Support for Pioneering Research Initiated by the Next Generation (SPRING) under Grant JPMJSP2145, and the National Natural Science Foundation of China (Grant Nos. 62072483 and 62203238).

Y. Song, K. Wang, Z. Lei and S. Gao are with the Faculty of Engineering, University of Toyama, Toyama-shi, 930-8555, Japan. (E-mail: sytforwork@gmail.com; greskofairy@gmail.com; leizystu@outlook.com; gaosc@eng.u-toyama.ac.jp).

Z. Zhan is with the College of Artificial Intelligence, Nankai University, Tianjin 300350, China. (E-mail: zhanapollo@163.com).

The primary goal of EC is to optimize a given objective function and identify the optimal solution within defined constraints [3]. Nevertheless, when dealing with extensive representational data like images and text, EC approaches often face limitations and overpower. Therefore, deep learning (DL) was introduced to tackle these daunting tasks in the early 21st century. DL has witnessed remarkable success, particularly in computer vision, natural language processing, image and speech recognition, etc [4]. Deep neural networks are trained to automatically update connection weights to achieve better learning results and handle tasks more effectively [5]. The rapid advancement of DL is driven by factors such as large-scale datasets, enhanced hardware capabilities, and improved algorithms, positioning it as a key driving force in the field of artificial intelligence [6], [7].

Although EC and DL have different tasks and goals, the interplay between the two branches has been a subject of keen scholarly interest. A superior DL model can automatically fine-tune neural network weights and parameters to achieve desired outcomes. Meanwhile, EC is geared toward exploring solution spaces to discover optimal solutions without an inherent learning process [8]. Borrowing EC to evolve the topology of neural networks or using neural networks to empower learning capabilities within EC are exciting sources of inspiration. Therefore, researchers have aimed to unify EC and DL to harmonize and complement each other to generate a more powerful artificial intelligence tool [9].

Neuroevolution stands out as a successful approach to optimizing artificial neural networks among so many combinatorial attempts. It utilizes evolutionary algorithms or some EC techniques to train various aspects of neural networks, including activation functions, hyperparameters, neural architectures, etc. Initially, researchers endeavor to investigate whether evolutionary algorithms could replace backpropagation to evolve a network's weights and hyperparameters [10]. Configuring reasonable hyperparameters faces difficulties such as different types of variables, large search scale space, and high evaluation costs. To address these challenges for convolutional neural networks, Li et al. [11] propose a novel hybrid distribution estimation algorithm. This algorithm employs hybrid variable hyperparameters encoding coupled with an orthogonal initialization strategy, effectively managing the extensive search space. Notably, it excels in hyperparameter optimization, as evidenced by its robust performance in medical diagnostics. Owing to the conclusion that evolutionary methods can be effectively applied to optimize network topology and connection weights, scholars challenge whether reasonable neu-

ral architectures can be searched directly with evolutionary algorithms. The process of constructing a suitable target architecture requires a great deal of experimentation and trial-and-error due to the multitude of structural variables and the significant computational overhead. Neural architecture search can effectively jump out of the inherent thinking paradigm and allow algorithms to compute complex and effective topologies automatically. Chen et al. [12] integrate reinforcement mutation strategies into neural architecture search, steering the efficient evolution of populations in the model to evolve to a better state with fewer computational resources. The reinforced evolutionary neural architecture search achieves competitive accuracy and semantic segmentation performance on multiple extensive datasets.

The integration of evolutionary algorithms with neural networks has been validated through various examples, demonstrating their ability to support or even execute neural network functions and tasks effectively. In recent years, scholars have been keen to imbue evolutionary populations with the concept of learning ability. Nevertheless, most attempts to incorporate learning models into evolutionary algorithms have performed poorly, hindered by the dual black-box nature of neural networks and optimization processes. This impasse is broken with the emergence of transfer learning, which adeptly introduces a learning dimension into EC [13]. Transfer learning leverages knowledge gained from one problem to expedite solutions to related issues, transferring insights based on function evaluation outcomes to either homogeneous or heterogeneous search spaces. In a parallel vein, reinforcement learning, with its exploratory approach, augments the search prowess of evolutionary algorithms through iterative trial and error. These concepts of learning and paradigms of problem-solving exhibit remarkable compatibility when integrated into the algorithmic processes of solving, searching, and evolving.

The integration of evolutionary algorithms with neural networks collectively represents a burgeoning field, poised to mitigate inherent limitations in both domains. This synergy heralds a promising trajectory for the refinement of evolutionary algorithm exploration and exploitation, as well as the enhancement of neural networks. The predominant emphasis in current research on the integration of evolutionary algorithms and deep learning is the optimization of algorithmic hyperparameters. Additionally, some studies concentrate on leveraging surrogate models to emulate abstract search spaces [14], facilitating effective exploration within broader search domains. This emphasis contributes to the improved sampling efficiency of algorithms within the expansive search space. There is a noticeable lack of attention directed towards modeling the authentic search space to extract greater solutions. Hence, we investigate the feasibility and impact of the network in reconstructing the authentic solution space on the population.

Fig. 1 provides a visual representation of utilizing a neural network to reconstruct the search space based on the sampling results of evolutionary operators within the authentic search space, aiming to obtain higher-quality solutions. The points in the figure represent the sampling results of the algorithm in the search space. The results of the network reconstruction are shown on the right side of the figure, with colors ranging from red to blue. The bluer color denotes the higher quality of solutions. While the algorithm samples discretely within the search space, the results by network reconstruction become continuous. Observably, the network demonstrates the capability to discern solutions of superior quality within the search space through the reconstruction of algorithmically sampled results. In this paper, we propose a deep learning-based search space reconstruction evolution strategy algorithm (DLES) that harnesses the formidable fitting capabilities of neural networks to reconstruct the authentic search space, thereby enhancing the quality of the solution. Distinguishing itself from other methods that incorporate neural networks in EC, our approach acquires superior individuals directly from the network based on its reconstruction results to the search space. To the best of our knowledge, we are the first to employ deep learning for reconstructing the authentic search space. The main contributions of this work include the following:

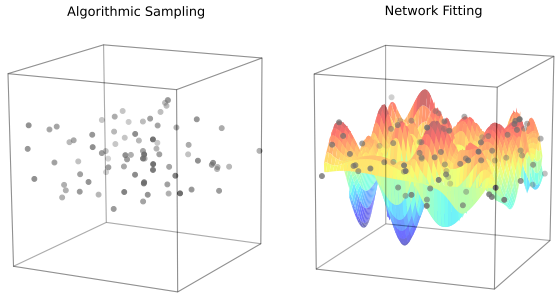

Fig. 1. Visualization of algorithm sampling and network reconstruction.

1) Innovatively, we propose a novel DLES algorithm, it employs a novel full-connected neural network to reconstruct the search space. The network utilizes the fitness as its input to construct individual information as the output. It is noteworthy that the input dimension of the network is set to one, deviating significantly from conventional network architectures.
2) Initially, we design the neural network to address a discrete sample of solution space. It vastly extends the sample, making the algorithm gain a more nuanced understanding of the search space, thereby assisting in the discovery of superior solutions.
3) We perform massive experiments to verify the performance of DLES on diverse CEC test sets, compared with state-of-the-art algorithms, which demonstrate DLES's superiority. Besides, we comprehensively analyze the impact of hyperparameters and space reconstruction networks to achieve stable and robust performance.

Section II gives a preliminary about the evolution strategy and the related work of learning evolutionary computation. Section III details the DLES. Section IV illustrates the experiments with results and analyses. Finally, the conclusion is provided in Section V.

II. Preliminaries

This section introduces the evolution strategy and the related work of learning evolutionary computation.

A. Evolutionary Computation

EC is an essential tool in the field of optimization, known for their ability to solve complex and nonlinear problems that are often beyond the reach of traditional optimization methods. EC is inspired by the principles of natural evolution and includes a variety of algorithms such as genetic algorithm, evolution strategies, differential evolution, etc. These algorithms are designed to mimic the process of natural selection, where a population of candidate solutions evolves over time, guided by fitness functions that measure the quality of solutions. Through mechanisms like selection, crossover, and operators, evolutionary algorithms explore the solution space and converge towards optimal or near-optimal solutions.

Evolutionary algorithms also, represent a broader class of optimization techniques, which are designed to provide high-quality solutions to complex problems by combining heuristic methods with strategies for escaping local optima, thus balancing exploration and exploitation of the search space [15]. They are particularly effective for problems where the search space is vast, rugged, or poorly understood, making them versatile tools for a wide range of applications.

Evolutionary algorithms have been successfully applied to various domains, including engineering design, logistics, scheduling, machine learning, and bioinformatics. Their ability to handle complex, multi-objective, and dynamic optimization problems has made them a popular choice among researchers and practitioners [16]. However, the design and implementation of effective metaheuristic algorithms require a deep understanding of both the problem domain and the algorithm's behavior, as well as careful tuning of parameters to achieve the best performance.

In conclusion, evolutionary algorithms provide a powerful and versatile tool for solving complex optimization problems. Their ability to find good solutions in challenging search spaces makes them invaluable in a wide range of applications. However, the successful application depends on a careful balance of exploration and exploitation, proper parameter tuning, and, in some cases, the integration of problem-specific knowledge to enhance performance.

B. Evolution Strategy

Evolutionary strategy (ES) is a heuristic technique for black-box optimization, widely applied across various disciplines and engineering domains [17], [18]. The ES aims to simultaneously generate solution vectors with the advantage of fewer evaluations and computational resources. The design principles of ES are unbiasedness, randomization, and adaptive control over samples [19]. The most widely used is the covariance matrix adaptation evolution strategy (CMAES) due to its rotational invariance, making it effective for targeting unimodal continuous optimization problems [20]. CMAES obtains the covariance matrix C through matrix decomposition and transformation in the search space, aiding in making the objective function separable for efficient solving. CMAES updates the covariance matrix by evolving the weighted maximum likelihood estimates for the selected directions. This updating process can be expressed as [21]:

$$C_{g+1} = (1 - c_1 - c_c)C_g + c_1 P_{g+1} P_{g+1}^T + c_c \sum_{i=1}^{\lambda} \Gamma_i, \qquad (1)$$

where g denotes the number of iterations; c_1 and c_c represent the scale transformation rates of the covariance matrix; λ is the number of promising individuals in the top λ rankings. P is called the evolutionary path and denotes the accumulation of the mean of the historical distribution. Γ_i denotes the mutation according to the Gaussian variational distribution for the ith individual. The original CMAES faces challenges of computational expense and sensitivity to boundary constraints. Moreover, its covariance matrix struggles to capture variable relationships in multimodal search spaces [22]. Numerous variants have emerged to address these issues, with the hybrid sampling evolution strategy (HSES) standing out as the CEC 2018 winner [23]. HSES introduces a hybrid sampling strategy that mitigates CMAES limitations by leveraging the robustness of univariate sampling in handling multimodal problems.

Another popular approach is the family of natural evolution strategies (NES). This family encompasses a group of gradient estimation algorithms associated with specific types of distributions. NES involves evaluating the fitness function at each sample point and computing gradients in the direction of higher expectations. Natural gradients excel in overcoming challenges like unstable performance and premature convergence. Wierstra et al. [24] explore NES and discover that the enhanced NES exhibits strong parallelism, excellent data processing efficiency, and effective exploration.

To conclude, ES focuses on mutation and operations and generally does not employ crossover operation, making the optimization process more stable, particularly when addressing multimodal or high-dimensional problems. Since ES does not depend on population recombination, it is less sensitive to the choice of the initial population, exhibiting strong robustness. Even with a low-quality initial solution, the algorithm can gradually converge to an optimal solution. ES is inherently designed for continuous optimization problems in high-dimensional spaces, especially those involving noise, nonlinearity, or complex multimodal landscapes. Given these advantages, we utilize evolution strategies, a fundamental EC method, to explore the solution space without introducing other complex mechanisms.

C. Learning-based Evolutionary Computation

In recent years, the challenges of incorporating learning abilities into evolution algorithms have been steadily addressed with the emergence of various learning models and the advancements in neural networks. Scholars have actively explored ways to integrate and enhance neural networks across various aspects of algorithmic structures, operators, and search patterns, resulting in significant successes. Currently, there are two main directions for modifying evolutionary algorithms through learning. The first direction involves employing learning models, such as transfer learning and reinforcement

learning. These bio-inspired learning models naturally align with different processes of EC, effectively enhancing the optimization and generalization capabilities of the algorithms [25]. The second involves directly embedding neural networks into the evolutionary search process. This integration assists the evolutionary population in decision-making, optimization, learning, and other operations. This dual approach reflects the ongoing efforts to synergize evolutionary algorithms with the power of learning models, marking a promising advancement in the field.

Evolutionary transfer optimization has emerged as a promising approach in the EC community [26]. This method involves transferring knowledge, such as population trajectories and individual information, from a historical task or an ongoing optimization task to address relevant new problems. This cross-domain transfer learning approach has been successful in complex optimization and has developed branches such as sequential optimization, multi-task optimization, etc. Jiang et al. [27] propose a novel algorithmic framework that integrates streaming transfer learning and memory mechanisms. By leveraging historical experience, this framework enables the population to learn the optimal potential space, significantly enhancing optimization speed without compromising solution quality. Qiao et al. [28] develop an evolutionary multi-task model to solve constrained multi-objective problems. The model establishes three tasks, namely the main search task, the global auxiliary task, and the local auxiliary task, facilitating efficient population search through the transfer of information between different tasks.

With the successful application of transfer learning in optimization, the integration of reinforcement learning into the EC framework has aroused enthusiasm, resulting in numerous reinforcement learning-assisted evolutionary algorithms. A distinctive feature of reinforcement learning is its utilization of an agent learning process grounded in Markovian decision-making for maximizing rewards [29]. The integrated approach generally determines the states, actions, and rewards in the decision chain based on the design goals of the algorithm and acts as an auxiliary regulator of the algorithm's search strategy. Sun et al. [30] innovatively incorporate a deep neural network as a controller for control parameters, coupling it with a reinforcement policy gradient algorithm to optimize the best parameters. This approach dynamically provides optimal parameter settings tailored to match the characteristics of test functions. Zhang et al. [31] propose a hybrid adaptive parameter control framework employing Q-learning and deep Q-learning to regulate agents. This self-switching learning model significantly enhances the solution quality of the original algorithm, verifying that empowering the algorithm to learn is a catalyst for optimization. Chrabaszcz et al. [32] demonstrate that ES serves as a viable alternative to reinforcement learning algorithms in addressing a range of challenging deep reinforcement learning problems.

In addition to the previous embedding of learning models, neural networks have been directly involved as trainers in EC in recent years. Ismayilov and Topcuoglu [33] propose a dynamic multi-objective optimization algorithm based on artificial neural networks, showcasing effective performance on dynamic workflow scheduling problems. This algorithm utilizes a multi-layer neural network to train paired solutions output, thereby grasping high-quality solutions. Zhan et al. [34] introduce a network-assisted learning-evolutionary algorithm, providing the evolutionary operator with the ability to learn and generate promising next-generation individuals. This learning operator learns from numerous successful solutions and training to enhance the entire evolutionary search process. Despite the challenges posed by neural networks simulating the intricate situation of black-box optimization and the trajectory of evolutionary populations, the training and learning results often show some limitations. There remains significant potential to explore in this direction, and maximizing the benefits of neural networks in evolutionary computation presents a compelling and intriguing topic.

III. The proposed methods

The section describes the motivation and the details of DLES. The proposed DLES reconstructs the authentic search space with a deep learning network, enhancing the algorithm's capability to navigate high-dimensional search spaces. This assists the algorithm in locating more optimal solutions.

A. Motivation

Evolutionary algorithms and neural networks are powerful tools for solving black-box problems. Combining them leverages their strengths to better handle complex problems and gain deeper insights into high-dimensional search spaces [35].

High-dimensional search spaces pose a major challenge, making direct simulation impractical. Surrogate-assisted evolutionary methods for dimensionality reduction face difficulties in capturing all features of high-dimensional spaces and require substantial computational resources, making them costly [14], [36], [37]. A more practical approach is to partition the solution space first and focus on selectively addressing key regions within it [38], [39].

During the search for the optimal individual, evolutionary algorithms follow two main stages: exploration and exploitation [40]. The success of the exploration phase is crucial for the effectiveness of the subsequent exploitation. While evolutionary algorithms sample the search space to infer its structure, the discrete nature of the sampling process makes it difficult to obtain a complete and accurate representation. This limitation may cause the algorithm to miss valuable areas and become trapped in local optima.

To address this challenge, we propose the DLES, leveraging the powerful non-linear fitting capability of neural networks to reconstruct these exploration paths. Exploration paths are the sequences of sampled points generated by the algorithm as it explores the solution space. By reconstructing these paths, DLES captures and utilizes the valuable information embedded within the exploration process. While the algorithm explores the solution space extensively, the exploration path occupies a significantly smaller portion compared to the full space. By focusing on reconstructing the exploration path, we substantially reduce the workload for network reconstruction, enabling us to complete the task with less complex networks.

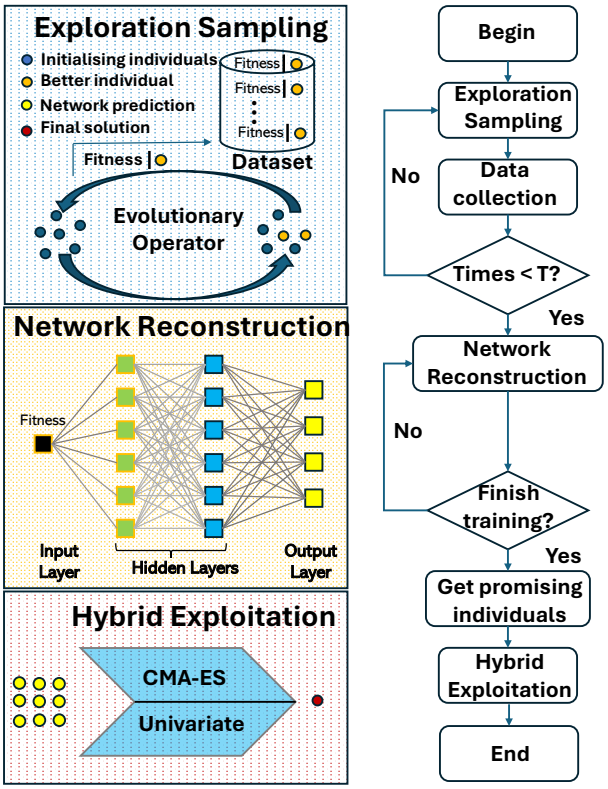

Fig. 2. The structure and flow framework of DLES.

The goal is to discover high-quality areas overlooked by the algorithm due to insufficient exploration. By reconstructing the paths, we can achieve a more effective fit for authentic high-dimensional search spaces, providing the algorithm with a better potential area and reducing the risk of falling into local optima [41].

B. Deep Learning-based Search Space Reconstruction Evolution Strategy

Building upon the concept of deep learning-based search space reconstruction empowers evolution algorithms for optimization, we introduce DLES. The overall structure and flow framework of DLES is depicted in Fig. 2. These components are executed in a sequential manner, comprising three distinct phases. The first phase, the evolutionary operator for exploration, explores the solution space and performs data collection and processing. In the subsequent phase, network reconstruction, the collected data is employed to reconstruct the solution space, thus facilitating the identification of more promising solutions for further exploration. The final phase, hybrid sampling for exploitation, leverages the network-generated solutions to execute the final optimization process.

1) Exploration Sampling: The primary focus of this stage involves conducting a thorough exploration of the search space using an evolutionary operator sampling and systematically collecting a dataset. The goal is to facilitate comprehensive exploration by the algorithm, capturing a diverse set of highly representative data points to construct a dataset suitable for network learning. This approach ensures the effective reconstruction of the search space.

2) Network Reconstruction: After collecting a feasible dataset, a neural network reconstructs the authentic search space, revealing solutions with higher potential. The network extensively leverages the exploration of sampled datasets for profound learning and fitting. Upon concluding the reconstruction process, the network can generate solutions with elevated exploitation possibilities rooted in the collected dataset.

3) Hybrid Exploitation: As the final stage of the DLES, a hybrid sampling strategy is employed to exploit the solutions generated in the network predictions. This method amalgamates the benefits of both single-sampling and multi-sampling, augmenting search efficiency and result accuracy.

C. Exploration Sampling

In the exploration section, DLES utilizes traditional evolutionary operators, exploring the search space through sampling and gathering pertinent information to construct a dataset suitable for neural network learning.

First, define the evaluation function $F(\cdot)$ based on a optimization problem, initialize the population P^0, and set the current ($NFEs = 0$). During the exploration process, the population undergoes a cumulative total of T iterations. The population P^t at generation t comprises N individuals, each denoted as y_i^t, and each individual has a dimension of D. DLES employs an evolutionary operator $\rho(\cdot)$ to generate M new individuals, forming the candidate set $P^{t'}$. Evaluating all candidates in $P^{t'}$ costs the evaluation function $F(P^{t'})$ to obtain the fitness values $\mathcal{F}$. Based on the fitness values, N individuals are selected from $P^{t'}$ to constitute the new population P^{t+1}. By continually updating the population through the utilization of evolutionary operators, DLES samples within the solution space and then comprehensively explores it. Simultaneously, it gathers relevant information, forming a *Dataset* conducive to neural network learning.

Due to the intrinsic characteristics of evolutionary operators in the sampling process, the algorithm iteratively endeavors to optimize specific locations for the discovery of improved solutions. However, this iterative procedure may lead to generating a substantial amount of redundant information. To enhance the efficiency and effectiveness of network reconstruction, DLES records the information of the current best individual (y_{best}) and its corresponding fitness value (f_{best}) upon detecting a decrease in fitness. This means that the algorithm updates its records only when a superior solution is identified compared to the previous best. By doing so, DLES ensures that it consistently retains the most optimal solution encountered throughout the search process. This information forms a dataset available for network learning. This decision avoids introducing excessive redundant information, ensuring that the neural network not only learns valuable information but fits the exploration path well. Throughout this process, the operators persistently attempt, make errors, and learn, progressively accumulating an understanding of the solution space. Such iterative procedures contribute to the stepwise optimization of the population, providing valuable learning *Dataset* for the neural network.

In DLES, an optimized univariate marginal distribution algorithm continuous (UMDAc) [42] is chosen as the evo-

Algorithm 1: Exploration Sampling

Input: Initialization: $P^0 \sim \mathcal{U}$, $P^0 \in R^{d\times N}$, $Dataset = \emptyset$, $NFEs = 0$, an evaluation function $F(\cdot)$ and an optimization function $\rho(\cdot)$

Output: *Dataset* and *NFEs*

```
F ← F(P^0);
select f_best from F;
NFEs+ = N;
select y_best from P^0 by f_best; //optimal individual
Add (y_best, f_best) in Dataset;
for t from 0 to T do
    P^t' = ρ(P^t), P^t' ∈ R^{d×M}, N < M;
    get y^t_best , f^t_best and P^{t+1} from sorted P^t'
    P^{t+1} = P^t'_i | i ∈ [1, N];
    NFEs+ = N;
    if f^t_best < f_best then
        Add (y^t_best, f^t_best) in Dataset;
        f_best = f^t_best;
    end if
end for
```

lutionary operator $\rho(\cdot)$. UMDAc exhibits high computational efficiency and operates with minimal interactions between variables. By sampling from the learned marginal distributions, UMDAc effectively balances exploration and exploitation, facilitating a thorough exploration of the search space. This balance is crucial for ensuring comprehensive coverage within the solution space. Additionally, UMDAc demonstrates robustness to noise and varying problem conditions, enhancing its effectiveness across diverse optimization scenarios [43].

This particular optimized UMDAc increases the weight of the best individual y_{best} when computing the mean μ. μ_d is to set the weighted sum of the best N solutions on each variable x_d,

$$\mu_d = \sum_{n=1}^{N} \omega_n x_{dn}. \tag{2}$$

The weight ω_n are determined by following equations:

$$\omega'_n = \ln(N+0.5) - \ln(n), \quad n \in [1, N] \tag{3}$$

$$\omega_n = \frac{\omega'_n}{\sum_{n=1}^{N} \omega'_n}. \tag{4}$$

The exploration pseudocode is presented in Algorithm 1.

D. Network Reconstruction

In the network reconstruction, a neural network utilizes data collected during the exploration process to reconstruct the authentic search space. Specifically, by learning from the sampled outcomes of evolutionary algorithms, neural networks can more directly and efficiently comprehend the structure of the search space. This capability enables neural networks to discover individuals situated along the exploration path, those that conventional evolutionary algorithms might overlook. These individuals exhibit greater exploitation potential.

In this research, we employ neural networks to reconstruct the paths in the search space by reverse deducing fitness value f, thereby providing an individual (y) with enhanced developmental potential. Specifically, the neural network takes f as input and predicts individual information ($\hat{y}$) in a D-dimensional space. The network comprises three fully connected layers, it is formulated as follows:

$$\begin{aligned} net^{(in)} &= W^{(in)} f + b^{(in)} \\ net^{(h)} &= tanh(W^{(h)} net^{(in)} + b^{(h)}) \\ net^{(out)} &= W^{(out)} net^{(h)} + b^{(out)}, \end{aligned} \tag{5}$$

where W and b represent the weights and bias of networks, respectively. $net^{(in)}$, $net^{(h)}$, and $net^{(out)}$ represents the output of the input, hidden, and output layer. Notably, the network may include a different number of hidden layers. In hidden layers, the hyperbolic tangent function (*tanh*) is employed as the activation function,

$$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \tag{6}$$

This function excels at handling negative data, producing output within the range of $[-1, 1]$. This characteristic makes it approach linearity near the zero point, contributing to mitigating the issue of gradient vanishing. The input layer has a size of 1, the output layer has a size of D. Through hidden layers, the neural network maps the input fitness value to a high-dimensional space and then projects the high-dimensional results back to the corresponding D-dimensional space.

During the training phase, the neural network optimizes the weight matrices and bias terms to minimize the difference between ($\hat{y}$) and y by the loss function. After training completion, the neural network, by fitting the information of the search space, can predict individual information corresponding to the input fitness value. When provided y with the smaller fitness minimum one, the neural network's task is to offer individuals with greater potential based on the reconstructed search space information.

Regarding the loss function, mean squared error (MSE) is generally considered a reasonable option. However, we observed that network reconstruction faces significant challenges when relying on MSE. In the context of fitting with the MSE loss function, the loss in a network arises from the mean square error between the network predictions and the actual labels. This implies that the network assigns equal importance to all data points, leading to significant consumption of computational resources and frequently failing to attain an optimal fit. During the process of training the network for reconstruction, our expectation is for the network to focus on regions with better fitness quality while minimizing attention to areas with poor solution quality. Therefore, we adopt a newly designed fitness mean squared error (FMSE) loss function, the formula for which is presented below:

$$loss = \frac{1}{D} \sum_{i=1}^{D} |y_i - \hat{y}_i|^2 + D\frac{|f_i - f_{mean}|}{f_{std}}. \tag{7}$$

Due to the difference in units between the MSE and fitness components, there is a risk that the network may overlook the MSE part when the fitness values are excessively large. To address this, we normalize the fitness values to follow a normal

Algorithm 2: Network Reconstruction

Input: $Dataset$, f_{best}
Output: A network-fitted optimal solution y^*

Randomly initialize network parameters;
for e **from** 0 **to** $Epoch$ **do**
  for pair in $Dataset$ **do**
    $f, y \leftarrow$ pair;
    $\hat{y}$ = network(f);
    calculate $loss$ by (7);
    update the network's parameters by $loss$;
  end for
end for
y^* = network(f_{best});

distribution. To further ensure consistency between the two components and prevent significant numerical discrepancies, we multiply the normalized fitness values by the dimension D, thereby aligning the units of both components.

The FMSE incorporates fitness into the computation, directing the network's attention toward regions with higher solution quality. By assigning lower loss values to regions with higher qualitative, the FMSE encourages the network to engage in more refined learning and optimization in these areas. The adoption of the FMSE allows the network to focus more on solution spaces with elevated fitness, thereby enhancing reconstruction performance and reducing computational resource consumption. This approach provides a novel pathway for achieving more efficient network training and better fitting within solution spaces. The network reconstruction pseudocode is presented in Algorithm 2.

E. Hybrid Exploitation

In the exploitation section, we adopt a hybrid sampling approach using CMAES combined with univariate sampling [43]. CMAES is particularly suitable for addressing unimodal problems, while single sampling is effective for handling multimodal problems. This combination combines multivariate sampling with univariate sampling to effectively enhance the algorithm's search performance across a wide range of problem types. Initially, CMAES is utilized to optimize the network-fitted optimal solution y^*, if multiple updates with CMAES yield no improvement in solutions, the execution of CMAES ceases and we obtain the optimal solution y'. Subsequently, y' is ues to generate a new population, subsequently passed to univariate sampling for additional optimization. For univariate sampling, the population updates by the mean (μ_d) and variance (σ_d) for each dimension. In order to guarantee the convergence of the algorithm, we set the parameter cc to control the σ_d, with cc approaching but remaining below 1 ($cc \rightarrow 1^-$) [23]. The exploitation pseudocode is presented in Algorithm 3.

Compared to traditional evolutionary algorithms, DLES exhibits significant advantages. The combination of evolutionary operators and deep learning allows these two methods to complement each other. In contrast to approaches that integrate deep learning, DLES possesses its own unique features. Many existing methods leverage deep learning to optimize algorithm hyperparameters or assist in selecting offspring. However, DLES utilizes neural networks to reconstruct the authentic search space. This approach not only enhances the algorithm's understanding of high-dimensional search spaces but also enables better utilization of information within the solution space, further improving algorithm performance.

Algorithm 3: Hybrid Exploitation

Input: A network-fitted optimal solution y^*, an evaluation function $F(\cdot)$ and $NFEs$;
Output: A development solution y^*

$y' \leftarrow$ CMAES(y^*, $F(\cdot)$);
$t = 0$;
Generate a total population P' containing M individuals based on y';
while not stopping criteria **do**
  select $N < M$ promising solutions P^t sorted as y_1 to y_N from P', record the best solution y_1 to y_{best};
  Calculate the weighted mean value μ_d and the standard deviation σ_d on each dimension y_d;
  if y_{best} no improvement in the latest I iterations **then**
    reduce the search space by:
    $\sigma_d = \sigma_d \times cc$ ($cc \rightarrow 1^-$);
  end if
  Sample P^t to generate M new solutions P' with variable σ_d and μ_d;
  $t = t + 1$;
end while
$\mathcal{F} \leftarrow F(P^{t+1})$;
select y^* from $\mathcal{F}$;//best individual

IV. Experiments and Analysis

To evaluate the DLES, we perform extensive experiments comparing it with twelve leading algorithms, including both traditional evolutionary methods and those with deep learning integration. We explore network structure and hyperparameters to determine optimal settings for DLES. Ablation studies assess the impact of omitting network reconstruction, highlighting the benefits of this approach. Additionally, we analyze DLES's role in population updating and begin to explore its complexity.

For a more detailed analysis, including the network effectiveness analysis, the performance of DLES on different types of problems, the impact of different network structures and the selection of activation functions, post-hoc tests in BBOB, and additional supplementary experimental results, please refer to the Supplementary File.

A. Benchmark Problems and Performance Metrics

Ensuring fair comparisons among different algorithms and fostering consensus on algorithm performance is paramount, so we utilize the single-objective real-parameter numerical optimization competition of IEEE Congress on Evolutionary Computation CEC 2014, CEC 2018, CEC 2022, and BBOB comprising a total of 95 problems as our benchmark test set.

TABLE I
COMPARISON SCORES AND RANKINGS OF DLES AND OTHER COMPETITORS ON CEC BENCHMARKS.

		Learning-based algorithm			Traditional algorithm					
Benchmark	Dimension	DLES	LEO-PSO	QHSES	CMAES	DDEARA	EA4eig	HSES	LSHADE	TAPSO	MLGSA
CEC 2014	D = 10	4.0500	9.7667	4.1833	7.5000	7.0667	**2.5833**	4.1167	2.9167	6.1500	6.6667
	D = 30	3.5333	9.2333	3.6333	6.7667	8.1333	3.5167	3.7333	**3.2333**	7.1167	6.1000
	D = 50	**3.1333**	8.7333	3.6667	6.5667	8.6333	3.8333	3.2167	4.1333	6.7833	6.3000
	D = 100	**2.8333**	8.1333	3.6667	5.9000	8.5333	4.7000	3.5333	4.4333	6.7833	6.4833
CEC 2018	D = 10	4.3621	7.8276	5.1897	8.3103	3.4483	**2.2759**	5.1897	3.7241	7.1897	7.4828
	D = 30	**2.7586**	9.0345	3.7241	7.9138	6.3621	3.8103	3.1207	3.3103	7.5172	7.4483
	D = 50	**2.6552**	8.8621	2.9310	7.2759	7.0517	4.4138	3.0345	3.6379	7.3103	7.8276
	D = 100	**2.2069**	8.8966	2.3793	6.8448	7.5000	5.0345	2.5172	4.3448	7.4483	7.8276
CEC 2022	D = 10	2.4167	9.0833	3.0000	5.1667	7.5833	**1.9167**	2.9167	7.7500	6.1667	9.0000
	D = 20	2.7083	8.2500	2.8750	5.3333	7.7500	**1.9583**	2.9583	8.1667	6.1667	8.8333
Average Score		**3.0657**	8.7820	3.5249	6.7578	7.2062	3.4042	3.4337	4.5650	6.8632	7.3969
Total Ranking		1	10	4	6	8	2	3	5	7	9

TABLE II
COMPARISON SCORES AND RANKINGS OF DLES AND OTHER COMPETITORS ON BBOB BECHMARK.

	DLES	a-CMAES	sc-CAMES	CMAES	HSES	E-WOA
D = 2	**1.5833**	4.3958	3.2292	3.2708	4.5625	3.9583
D = 3	**1.4583**	3.4792	3.1042	3.3542	5.1458	4.4583
D = 5	**1.7917**	2.9375	3.1458	2.8125	5.5208	4.7917
D = 10	**1.8125**	3.0208	3.0625	2.8958	5.6042	4.6042
D = 20	**2.0208**	2.5625	3.1458	2.8542	5.7292	4.6875
D = 40	**1.9375**	2.2500	3.0417	3.2083	5.8333	4.7292
Average Score	**1.7660**	3.1076	3.2881	3.0659	5.3993	4.5382
Total Ranking	1	4	2	3	6	5

The CEC benchmark test set of optimization problems covers a range of types, including unimodal, multimodal, hybrid, and more intricate hybrid problems. The bounds for each dimension are uniformly set between -100 and 100. In detail, CEC 2014 presents 30 distinct problems, while CEC 2018 contributes 29 problems. The test instances are further categorized into dimensions of 10, 30, 50, and 100. For these problems, the maximum number of function evaluations (*MFEs*) is set at $10{,}000 \times D$, where D represents the problem dimension. Moving on to CEC 2022, which introduces 12 problems categorized into dimensions of 10 and 20, the *MFEs* are set at 200,000 for D = 10 and 1,000,000 for D = 20. According to the CEC official guidelines, results less than 10^{-8} are recorded as 0, indicating that the algorithm successfully finds the global optimum.

The BBOB benchmark set includes a diverse array of optimization problems, specifically designed to assess algorithm performance across various complexities. Unlike the CEC benchmarks, the maximum number of function evaluations *MFEs* in BBOB is set 10^5 for all problem instances. The BBOB set comprises 24 problems, evaluated across dimensions of 2, 3, 5, 10, 20, and 40. This standardized evaluation limit ensures consistency in comparing algorithm performance across all problems and dimensions within the BBOB suite. On the BBOB set, we have retained the full precision of the values without truncating to zero.

To objectively assess and compare the experimental results of different algorithms, we employ the Wilcoxon rank-sum test as the comparative method. This rank-sum test statistical approach offers stability and flexibility, with relatively lenient assumptions about data distribution, making it well-suited for this research needs. We evaluate the differences between algorithms at a significance level of 0.05 [44]. By utilizing this statistical method, we can more accurately assess and compare the performance of different algorithms, leading to more objective conclusions.

To comprehensively compare the performance of different hyperparameters and network structures across the problem set, we further employ the Friedman rank-sum test [45] to rank the experimental results. For better validation of DLES performance, we utilize the Friedman rank-sum test to compute scores and rankings for the experimental results across all methods. We aggregate the performance by taking the average of the final results from multiple experimental runs before ranking, ensuring a robust comparison across different methods. This approach allows us to rank the performance of different hyperparameter configurations, thereby identifying the optimal hyperparameter settings. This method comprehensively considers multiple factors, providing relatively objective ranking results to assist us in making more informed decisions.

B. Experimental Setup

Empirically validating the effectiveness of our proposed method involves a comparison with twelve competitive algorithms, including classic CMAES [20], the improved versions of CMA-ES (a-CMAES and sc-CMAES [46]), the champion of CEC 2014 (LSHADE [47]), the champion of CEC 2018 (HSES [23]), the champion of CEC 2022 (EA4eig [48]), three widely adopted algorithms E-WOA [49], MLGSA [40], DDEARA [50], TAPSO [51], and two algorithms combining deep learning: QHSES [31] and LEO-PSO [34]. In DLES, hyperparameter configurations are as follows: number of updates to the population during the exploration process T = 100, network learning rate (lr) = 0.5, and the epoch is set to 10. The hyperparameter settings for each algorithm remain consistent with their respective original papers, detailed configurations can be found in the supplementary file.

This experimental workstation is equipped with Pytorch on a PC with an AMD Ryzen 7 1700X @ 3.4GHz, eight-core processor and 16GB RAM, using Ubuntu 18.04.6 LTS OS. Each algorithm is independently run for 51 times on each function to minimize variability and ensure the attainment of robust and consistent outcomes.

C. Effectiveness of Deep Learning-based Search Space Reconstruction Evolution Strategy

In Table I, we present the score results and final rankings of DLES and other methods on the benchmark. For each problem set, the Friedman rank-sum test is employed to calculate the algorithm's score on that specific set of problems. This score is then divided by the total number of problems to obtain the algorithm's average score across all problems within the current problem set. The average scores for all problem sets are summed up to derive the algorithm's final score across all problem sets. It is evident that DLES outperforms all comparative algorithms through the scores and ranking table. The specific results of the mean and standard deviation for the 51 experiments are detailed in the supplementary file.

In Table I, the performance of DLES across diverse test sets is distinctly depicted. Compared to EA4eig and LSHADE, DLES exhibited slightly diminished scores in dimensions 10 and 30 of CEC 2014. Nevertheless, it outperforms them in dimensions 50 and 100, securing the highest scores. Compared to DDEARA, EA4eig, and LSHADE, DLES records lower scores in dimension 10 of CEC 2018. However, it showcased exemplary performance in dimensions 30, 50, and 100, thereby securing the foremost position once again. In the CEC 2022, owing to reduced dimensions, DLES scores marginally below the leading algorithm EA4eig, ultimately securing the second position. The performance of DLES across diverse test sets accentuates its superiority in addressing high-dimensional problems. While it may trail behind certain algorithms in specific low-dimensional scenarios, its consistently remarkable performance underscores its potential in tackling high-dimensional challenges.

Table II presents the comparison scores and rankings of DLES against other competing algorithms on the BBOB benchmark across various dimensions. DLES consistently outperforms the other algorithms, underscoring its significant advantage in solving BBOB optimization problems. Overall, DLES achieves superior performance among all tested methods, demonstrating its effectiveness and robustness across different problem dimensions.

To further demonstrate the performance of DLES, we utilize the Wilcoxon rank-sum test, revealing its superiority over nine competitive algorithms across various dimensions on CEC 2018, as indicated by the Win/Tie/Loss ($W/T/L$) metrics. In Table III, DLES only lags behind a few powerful algorithms in low-dimensional problems when $D = 10$. Overall, DLES consistently outperforms other algorithms, showcasing its robust and effective optimization capabilities, with the dimensionality increasing an increasing number of wins and fewer losses. This trend highlights the algorithm's adaptability and robustness in tackling optimization challenges, especially in high-dimensional spaces. The total $W/T/L$ metrics across all dimensions further underscore DLES's exceptional performance, solidifying its position as a state-of-the-art algorithm that excels as the complexity of optimization problems escalates with higher dimensions. A details analysis of problem types is summarized in Supplementary File.

D. Discussion of Network Structure and Hyperparameters

We explore the architecture of neural networks and devise a series of network models for experimentation. To simplify the models and avoid the use of complex neural network structures, we employed only fully connected layers to delve into the impact of different widths and depths on the final reconstruction capability of the network. While maintaining consistency in the input and output layers, we designed the following five network structures:

1) Net1: 1-D,
2) Net2: 1-2D-D,
3) Net3: 1-2D-4D-D,
4) Net4: 1-2D-4D-2D-D,
5) Net5: 1-2D-4D-8D-D.

Net1 is a network without the hidden layer. Net2 – Net5 represent the network with a different number of hidden layers, where 2D, 4D, and 8D denote the size of each hidden layer. Simultaneously, to assess the performance of the proposed loss function, we compare the performance of the conventional MSE loss function with the proposed FMSE loss function across different network structures.

In Table IV, we present combinations of various network structures and loss functions, ranked according to the experimental results using the Friedman test to reflect their relative performance, where "None" means a variant without using deep learning. These experimental results indicate that the proposed FMSE loss function exhibits superior performance in network fitting. Additionally, different network structures also influence the performance of the loss function to some extent. By comparing the experimental results, we observe that the Net3 structure demonstrates superior performance when combined with the FMSE loss function.

We conduct the hyperparameters of DLES, involving discussions on the sampling iteration count in exploration (T), lr, and epoch. Experimental results for various combinations of hyperparameters are computed using the Friedman rank-sum test, and the corresponding scores are ranked, with lower scores indicating higher ranks. Table V presents the experimental results, revealing that the optimal hyperparameter combination is $T = 100$, lr = 0.5, and the epoch is set to 10.

TABLE III
WILCOXON RANK-SUM TEST RESULTS OF DLES AND OTHER COMPETITORS ON CEC 2018 IN FOUR DIFFERENT DIMENSIONS IN TERMS OF $W/T/L$.

DLES vs.	$D = 10$	$D = 30$	$D = 50$	$D = 100$	Total
LEO-PSO	29/0/0	29/0/0	28/0/1	28/0/1	114/0/2
QHSES	12/14/3	14/13/2	8/19/2	5/23/1	39/69/8
CMAES	24/3/2	26/2/1	23/3/3	20/4/5	93/12/11
DDEARA	9/5/15	23/4/2	26/0/3	27/2/0	85/11/20
EA4eig	4/9/16	18/6/5	21/4/4	25/2/2	68/21/27
HSES	12/16/1	8/21/0	5/23/1	6/22/1	31/82/3
LSHADE	7/10/12	13/11/5	19/7/3	23/6/0	62/34/20
TAPSO	24/3/2	28/1/0	28/1/0	28/1/0	108/6/2
MLGSA	26/1/2	27/1/1	28/0/1	28/0/1	109/2/5

TABLE IV
FRIEDMAN RANKING OF DLES ACROSS VARIOUS COMBINATIONS OF NETWORK STRUCTURES AND LOSS FUNCTIONS.

Net	Loss	rank	Net	Loss	rank
Net1	MSE	11	Net1	FMSE	9
Net2	MSE	8	Net2	FMSE	10
Net3	MSE	2	**Net3**	**FMSE**	**1**
Net4	MSE	3	Net4	FMSE	4
Net5	MSE	7	Net5	FMSE	5
None	None	6			

TABLE V
FRIEDMAN RANKING OF DLES UNDER DIFFERENT HYPERPARAMETER COMBINATIONS.

T	lr	epoch	score	rank	T	lr	epoch	score	rank
50	0.1	10	431.5	19	100	0.5	50	329	16
50	0.5	10	427	18	100	0.5	100	313.5	14
50	1.0	10	445	20	100	1.0	10	309	13
100	0.01	10	275	9	200	0.1	10	241.5	2
100	0.01	50	355	17	200	0.1	50	256.5	6
100	0.01	100	325	15	200	0.1	100	269.5	8
100	0.1	10	286.5	10	200	0.5	10	259	7
100	0.1	50	291	11	200	0.5	50	251.5	5
100	0.1	100	298	12	200	0.5	100	246.5	3
100	**0.5**	**10**	**231**	**1**	200	1.0	10	249	4

Additionally, the scores obtained through the rank-sum test indicate minimal differences in the final scores among different hyperparameter combinations, suggesting that the performance gap between these combinations is small. This observation implies that our proposed method is insensitive to hyperparameter variations, showcasing robustness. Consequently, our method demonstrates strong robustness when confronted with diverse hyperparameter settings, contributing to improved stability and reliability.

To better demonstrate the impact of different network structures on the DLES, we provide additional analyses of neural structure in the Supplementary File. These experiments compare the performance of the algorithm using Net1-Net5 networks versus not using any network (No-Net) across various types of problems. Overall, network-based fitting consistently outperforms cases without networks. Typically, as problem dimensionality and complexity increase, more complex networks tend to deliver better performance. While selecting different networks for specific problems could yield better results, such an approach would undermine the generality and scalability of the algorithm. Tailoring networks for individual problems increases complexity and reduces the robustness of the solution, which goes against the broader goal of developing a more universally applicable optimization method. Therefore, we choose NET3, which performs well across most problems, as the base network for DLES.

E. Algorithm Analysis of Exploration and Exploitation

In the aforementioned discussion of hyperparameters, we determine the optimal exploration sampling count as $T = 100$. In alignment with the concept of DLES, we aim for the neural network to reconstruct the exploration paths of the algorithm within the solution space. Therefore, we employ a dimension-wise diversity-based method [52] to gauge the relationship between exploration and exploitation. This relationship is measured in terms of the percentage of exploration and exploitation, and the equation can be expressed by the following:

$$XPL = \left(\frac{Div}{Div_{max}}\right), \tag{8}$$

$$XPT = 1 - XPL, \tag{9}$$

where XPL and XPT represent the level of exploration and exploitation, respectively. Div_{max} is the maximum distance of all Div in the current iteration.

$$Div = \frac{1}{D}\sum_{j=1}^{D} Div_j, \tag{10}$$

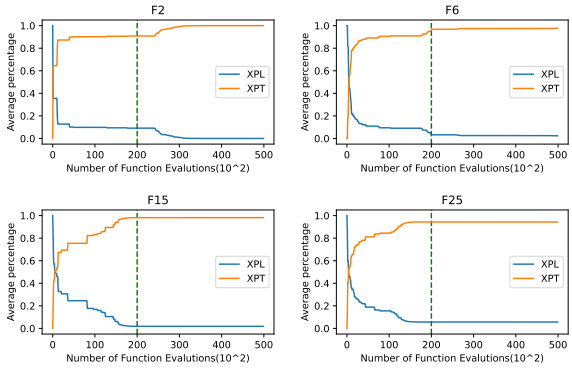

Fig. 3. Exploration and exploitation percentage of DLES for F2, F6, F15, and F25 with 30 dimensions.

$$Div_j = \frac{1}{N}\sum_{i=1}^{N}\left|x_i^j - median(x^j)\right|, \tag{11}$$

where x_i^j denotes the jth dimension of the ith individual. N and D represent the population and dimension size, respectively. $median(x^j)$ represents the median value of the jth dimension in the entire population.

Exploring and developing proportions within a population can reveal the current state of the population. In Fig. 3, we select representatives F2, F6, F15, and F25 to illustrate the population dynamics of DLES during the search process in the solution space on $D = 30$. The dashed vertical line in the figure corresponds to when DLES utilizes the network for reconstruction. The left side of the dashed line represents the exploration and sampling phase of our proposed DLES, while the right side represents the subsequent development phase. As the population state remains nearly constant during the subsequent development process, only the population state at the first 50,000 evaluations is displayed.

From Fig. 3, it is evident that when DLES employs the network for simulating the solution space, XPL's value approaches 0, XPT one approaches 1. The population has almost entirely transitioned to the development phase, signifying the conclusion of the exploration process. The data provided to train the network for reconstruction corresponds to the population's exploration path in the solution space, aligning precisely with the original intent of our algorithm design.

The quantity and quality of training samples are crucial for effective network learning; hence, ensuring high-quality adequate training sample is of paramount importance. In terms of sample quantity, the quantity is controlled by the optimal exploration sampling count T. Theoretically, increasing the sampling iterations deepens the understanding of the solution space. A higher T value results in more training samples, which enhances the network's performance. However, due to the limited number of evaluations, excessive allocation of evaluations to the exploration phase can lead to insufficient evaluations for subsequent algorithm exploitation. Therefore, a careful balance of T is required. Table V discusses the impact of various T. It can be observed that, with consistent network training parameters, the algorithm achieves the best overall performance when T is euqal to 100. If the sampling iterations are too few, the network struggles to learn effectively; if too

many, it adversely affects the subsequent exploitation phase of the algorithm.

Regarding sample quality, we indirectly ensure the quality of training samples by measuring the exploration-exploitation ratio of the population. When T is equal to 100, the evaluations consumed amount to 2000. As shown in Fig. 3, the population stabilizes at the same level, primarily remaining in the exploitation phase, indicating that the algorithm has largely completed its exploration phase. This stability ensures the quality of our training samples.

F. Complexity Analysis

DLES does not entail supplementary evaluation costs in its utilization of the network for learning and fitting. Consequently, it avoids incurring additional expenses in situations where constraints exist on the number of evaluations, rendering it particularly advantageous for problems characterized by limited evaluation resources.

In terms of space complexity, DLES differs from other algorithms as it necessitates additional storage for the dataset used in network training. However, the proportion of this data relative to the overall evaluation dataset is $1/K$, where K is a parameter associated with the hyperparameter of the sampling iterations T, the value of $1/K$ is exceptionally low at 0.0776. Therefore, the increase in space complexity for DLES due to network augmentation does not exceed $O(N \cdot D)$. This implies that, in comparison to other algorithms, DLES does not exhibit a significant increase in overall space utilization.

The DLES's time complexity is closely tied to the training of the neural network. Specifically, the increased time is primarily consumed during the process of network training. The time consumption for network training is associated with the number of training epochs. The time complexity of the DLES algorithm is expressed as $O(n) = T \cdot O(N \cdot D) + E \cdot \alpha \cdot T \cdot O(N \cdot D) + (G - T) \cdot O(N^2 \cdot D)$, where T is the number of updates to the population during the exploration process, G is the total number of iterations, E is the number of training epochs for the network, $\alpha \cdot T$ represents the number of iterations in each epoch and α belongs to the interval (0,1]. For our recommended parameter setting of the epoch is 10, the network increases time consumption does not exceed the $O(N \cdot D)$ range. It is essential to highlight that leveraging GPU acceleration during the neural network training process can further diminish the time required for training.

In Table VI, we depict the time consumption for a single run on the CEC 2018 problem set under scenarios without network, with network training using CPU, and with network training using GPU. In Table VI, we observe that the use of network training only increases the overall time consumption by approximately 14-18%. However, it is noteworthy that the total time consumption with GPU acceleration is slightly higher than that with CPU usage. We hypothesize that this might be attributed to the time required for data transfer from CPU to GPU, particularly evident in scenarios with smaller problem dimensions and lesser data volume. In such cases, the data transfer process constitutes a significant portion of the overall time. As the problem dimensions increase, the proportion of time spent on data transfer gradually diminishes. The data in Table VI largely aligns with our conjecture, in the case of $D = 100$, the GPU training accelerates the process.

TABLE VI
TOTAL RUNTIME FOR A SINGLE EXECUTION ON CEC 2018 ACROSS VARYING DIMENSIONS.

	$D = 10$	$D = 30$	$D = 50$	$D = 100$	Total time	Growth rate
NO Net	44.0648	315.8045	630.8677	**2522.8911**	3513.6281	-
IN CPU	85.1327	417.0270	752.5798	**2761.5299**	4016.2694	14.3%
IN GPU	157.7279	484.6854	784.5067	**2729.6929**	4156.6139	18.2%

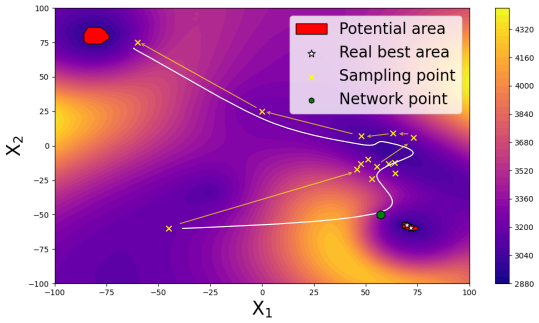

Fig. 4. Visualization of the exploration process of DLES algorithm on CEC 2018 F27.

In summary, DLES excels in complexity analysis, circumventing additional evaluation costs, ensuring effective space utilization, and enabling GPU acceleration for high-dimensional problems.

G. Visualization of Reconstructed Exploration Path

To better demonstrate the process and advantages of our proposed algorithm in reconstructing the solution space. We provide a two-dimensional visualization of the DLES algorithm's exploration process on CEC 2018 F27. As illustrated in Fig. 4, the red regions represent potential areas, while the white pentagrams denote the true optimal points. The yellow crosses indicate the sampling locations during the algorithm's exploration within the search space. The green hexagons show the positions of the solutions with higher potential provided after network reconstruction. The colors in the figure represent the fitness values, with darker colors indicating smaller fitness values. For the F27 problem, the optimal fitness value is 2800.

The figure illustrates the sampling process of the algorithm as it explores the solution space. It is evident that the results of multiple samplings form an exploration path through the solution space. This path is indicated by yellow arrows connecting consecutive sampling points. The white curve represents the exploration path reconstructed by the network based on historical sampling points.

The visualization reveals that the algorithm is misled and gradually converges towards a local optimum. However, the network's reconstruction of the solution space corrected this issue, allowing the algorithm to subsequently explore and ultimately find the true optimal solution.

V. CONCLUSION

In this paper, we propose a deep learning-based search space reconstruction evolution strategy algorithm (DLES) for reconstructing the authentic solution space. The algorithm leverages

a neural network to model exploration paths derived from the sampling results of evolutionary operators within the solution space. By doing so, the network identifies individuals with greater potential along the exploration path, guiding further search through a hybrid sampling approach. Unlike traditional evolutionary operator sampling methods, this network-based reconstruction offers a deeper understanding of the solution space, reducing the risk of local optima in high-dimensional problems. As a result, DLES more efficiently explores the solution space, uncovering promising solutions. To assess DLES's performance, we compare it to top algorithms on CEC and BBOB benchmarks, where it shows superior results. Experiments reveal that DLES is robust to hyperparameter changes, maintaining strong stability and generalization. We also examine the population state and the effects of network reconstruction, confirming DLES's ability to find more promising solutions. Finally, we analyze DLES's complexity, showing that the added time and space costs from network reconstruction are minimal, with no extra evaluation counts needed.

DLES gathers crucial insights from the entire search process by fitting exploration paths. While modeling solution spaces by using neural networks is not new, previous methods often perform poorly because they attempt to fit the entire solution space, which results in excessive network complexity. By focusing solely on modeling exploration paths, we significantly reduce the network's load, allowing even simpler architectures to handle complex tasks. The incorporation of neural networks, with their powerful nonlinear fitting capabilities, enables DLES to combine the strengths of both network-driven and algorithm-driven approaches, highlighting the value of integrating deep learning with evolutionary algorithms in optimization challenges.

In the future, we plan to extend the solution space reconstruction method to various aspects. Firstly, this paper primarily focuses on single-objective problems, and we hope to extend the DLES method to multi-objective problems. Secondly, by exploring and further developing more powerful evolutionary operators and strategies, we hope to enhance the potential of DLES. Finally, in this paper, we only reconstructed the exploratory part of the solution space. In the future, we will use more robust networks to reconstruct the more true exploration space. These extensions will contribute to further advancing the development of the DLES algorithm, providing more possibilities for its practical applications in real-world problems.

References

[1] A. E. Eiben and J. Smith, "From evolutionary computation to the evolution of things," *Nature*, vol. 521, no. 7553, pp. 476–482, 2015.

[2] B. Li, Z. Wei, J. Wu, S. Yu, T. Zhang, C. Zhu, D. Zheng, W. Guo, C. Zhao, and J. Zhang, "Machine learning-enabled globally guaranteed evolutionary computation," *Nature Machine Intelligence*, pp. 1–11, 2023.

[3] S. Gao, Y. Yu, Y. Wang, J. Wang, J. Cheng, and M. Zhou, "Chaotic local search-based differential evolution algorithms for optimization," *IEEE Transactions on Systems, Man and Cybernetics: Systems*, vol. 51, no. 6, pp. 3954–3967, 2021.

[4] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, no. 7992, pp. 570–578, 2023.

[5] C. Lee, H. Hasegawa, and S. Gao, "Complex-Valued Neural Networks: A Comprehensive Survey," *IEEE/CAA Journal of Automatica Sinica*, vol. 9, no. 8, p. 1406–1426, 2022.

[6] W. Ding, W. Pedrycz, G. G. Yen, and B. Xue, "Guest editorial evolutionary computation meets deep learning," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 5, pp. 810–814, 2021.

[7] G. Ma, Z. Wang, W. Liu, J. Fang, Y. Zhang, H. Ding, and Y. Yuan, "Estimating the state of health for lithium-ion batteries: A particle swarm optimization-assisted deep domain adaptation approach," *IEEE/CAA Journal of Automatica Sinica*, vol. 10, no. 7, pp. 1530–1543, 2023.

[8] Z. Lei, S. Gao, Z. Zhang, H. Yang, and H. Li, "A chaotic local search-based particle swarm optimizer for large-scale complex wind farm layout optimization," *IEEE/CAA Journal of Automatica Sinica*, vol. 10, no. 5, pp. 1168–1180, 2023.

[9] Y. Yu, Z. Lei, Y. Wang, T. Zhang, C. Peng, and S. Gao, "Improving dendritic neuron model with dynamic scale-free network-based differential evolution," *IEEE/CAA Journal of Automatica Sinica*, vol. 9, no. 1, pp. 99–110, 2022.

[10] E. Galván and P. Mooney, "Neuroevolution in deep neural networks: Current trends and future challenges," *IEEE Transactions on Artificial Intelligence*, vol. 2, no. 6, pp. 476–493, 2021.

[11] J.-Y. Li, Z.-H. Zhan, J. Xu, S. Kwong, and J. Zhang, "Surrogate-assisted hybrid-model estimation of distribution algorithm for mixed-variable hyperparameters optimization in convolutional neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 5, pp. 2338–2352, 2023.

[12] Y. Chen, G. Meng, Q. Zhang, S. Xiang, C. Huang, L. Mu, and X. Wang, "RENAS: Reinforced evolutionary neural architecture search," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4787–4796.

[13] W. Zhang, L. Deng, L. Zhang, and D. Wu, "A survey on negative transfer," *IEEE/CAA Journal of Automatica Sinica*, vol. 10, no. 2, pp. 305–329, 2023.

[14] C. He, Y. Zhang, D. Gong, and X. Ji, "A review of surrogate-assisted evolutionary algorithms for expensive optimization problems," *Expert Systems with Applications*, vol. 217, p. 119495, 2023.

[15] Q.-T. Yang, J.-Y. Li, Z.-H. Zhan, Y. Jiang, Y. Jin, and J. Zhang, "A hierarchical and ensemble surrogate-assisted evolutionary algorithm with model reduction for expensive many-objective optimization," *IEEE Transactions on Evolutionary Computation*, pp. 1–1, 2024.

[16] N. Li, L. Ma, G. Yu, B. Xue, M. Zhang, and Y. Jin, "Survey on evolutionary deep learning: Principles, algorithms, applications, and open issues," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–34, 2023.

[17] M. Xu, Y. Mei, F. Zhang, and M. Zhang, "Genetic programming for dynamic flexible job shop scheduling: Evolution with single individuals and ensembles," *IEEE Transactions on Evolutionary Computation*, 2023, doi: 10.1109/TEVC.2023.3334626.

[18] L. Si, X. Zhang, Y. Tian, S. Yang, L. Zhang, and Y. Jin, "Linear subspace surrogate modeling for large-scale expensive single/multi-objective optimization," *IEEE Transactions on Evolutionary Computation*, 2023, doi: 10.1109/TEVC.2023.3319640.

[19] S. Liu, Q. Lin, J. Li, and K. C. Tan, "A survey on learnable evolutionary algorithms for scalable multiobjective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 27, no. 6, pp. 1941–1961, 2023.

[20] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.

[21] B. Karmakar, A. Kumar, R. Mallipeddi, and D.-G. Lee, "CMA-ES with exponential based multiplicative covariance matrix adaptation for global optimization," *Swarm and Evolutionary Computation*, vol. 79, p. 101296, 2023.

[22] Z. Li and Q. Zhang, "Variable metric evolution strategies by mutation matrix adaptation," *Information Sciences*, vol. 541, pp. 136–151, 2020.

[23] G. Zhang and Y. Shi, "Hybrid sampling evolution strategy for solving single objective bound constrained problems," in *2018 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2018, pp. 1–7.

[24] D. Wierstra, T. Schaul, T. Glasmachers, Y. Sun, J. Peters, and J. Schmidhuber, "Natural evolution strategies," *The Journal of Machine Learning Research*, vol. 15, no. 1, pp. 949–980, 2014.

[25] Y. Yu, S. Gao, Y. Wang, and Y. Todo, "Global optimum-based search differential evolution," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 2, pp. 379–394, 2019.

[26] K. C. Tan, L. Feng, and M. Jiang, "Evolutionary transfer optimization - a new frontier in evolutionary computation research," *IEEE Computational Intelligence Magazine*, vol. 16, no. 1, pp. 22–33, 2021.

[27] M. Jiang, Z. Wang, L. Qiu, S. Guo, X. Gao, and K. C. Tan, "A fast dynamic evolutionary multiobjective algorithm via manifold transfer

learning," *IEEE Transactions on Cybernetics*, vol. 51, no. 7, pp. 3417–3428, 2021.
[28] K. Qiao, J. Liang, Z. Liu, K. Yu, C. Yue, and B. Qu, "Evolutionary multitasking with global and local auxiliary tasks for constrained multi-objective optimization," *IEEE/CAA Journal of Automatica Sinica*, vol. 10, no. 10, pp. 1951–1964, 2023.
[29] Z. Zhu, K. Lin, A. K. Jain, and J. Zhou, "Transfer learning in deep reinforcement learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11, pp. 13 344–13 362, 2023.
[30] J. Sun, X. Liu, T. Bäck, and Z. Xu, "Learning adaptive differential evolution algorithm from optimization experiences by policy gradient," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 4, pp. 666–680, 2021.
[31] H. Zhang, J. Sun, T. Bäck, Q. Zhang, and Z. Xu, "Controlling sequential hybrid evolutionary algorithm by Q-Learning," *IEEE Computational Intelligence Magazine*, vol. 18, no. 1, pp. 84–103, 2023.
[32] P. Chrabaszcz, I. Loshchilov, and F. Hutter, "Back to basics: benchmarking canonical evolution strategies for playing atari," in *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, 2018, pp. 1419–1426.
[33] G. Ismayilov and H. R. Topcuoglu, "Neural network based multi-objective evolutionary algorithm for dynamic workflow scheduling in cloud computing," *Future Generation Computer Systems*, vol. 102, pp. 307–322, 2020.
[34] Z.-H. Zhan, J.-Y. Li, S. Kwong, and J. Zhang, "Learning-aided evolution for optimization," *IEEE Transactions on Evolutionary Computation*, vol. 27, no. 6, pp. 1794–1808, 2023.
[35] X. Liu, J. Sun, Q. Zhang, Z. Wang, and Z. Xu, "Learning to learn evolutionary algorithm: A learnable differential evolution," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 7, no. 6, pp. 1605–1620, 2023.
[36] M. Cui, L. Li, M. Zhou, J. Li, A. Abusorrah, and K. Sedraoui, "A Bi-population Cooperative Optimization Algorithm Assisted by an Autoencoder for Medium-scale Expensive Problems," *IEEE/CAA Journal of Automatica Sinica*, vol. 9, no. 11, pp. 1952–1966, 2022.
[37] M. Cui, L. Li, M. Zhou, and A. Abusorrah, "Surrogate-assisted autoencoder-embedded evolutionary optimization algorithm to solve high-dimensional expensive problems," *IEEE Transactions on Evolutionary Computation*, vol. 26, no. 4, pp. 676–689, 2022.
[38] Y. Liu, J. Liu, and S. Tan, "Decision space partition based surrogate-assisted evolutionary algorithm for expensive optimization," *Expert Systems with Applications*, vol. 214, p. 119075, 2023.
[39] Q. Fan and O. K. Ersoy, "Zoning search with adaptive resource allocating method for balanced and imbalanced multimodal multi-objective optimization," *IEEE/CAA Journal of Automatica Sinica*, vol. 8, no. 6, pp. 1163–1176, 2021.
[40] Y. Wang, S. Gao, M. Zhou, and Y. Yu, "A multi-layered gravitational search algorithm for function optimization and real-world problems," *IEEE/CAA Journal of Automatica Sinica*, vol. 8, no. 1, pp. 94–109, 2021.
[41] M. Zhou, M. Cui, D. Xu, S. Zhu, Z. Zhao, and A. Abusorrah, "Evolutionary optimization methods for high-dimensional expensive problems: A survey," *IEEE/CAA Journal of Automatica Sinica*, vol. 11, no. 5, pp. 1092–1105, 2024.
[42] P. Larrañaga and C. Bielza, "Estimation of Distribution Algorithms in Machine Learning: A Survey," *IEEE Transactions on Evolutionary Computation*, 2023, doi: 10.1109/TEVC.2023.3314105.
[43] Y. Fu and H. Wang, "A univariate marginal distribution resampling differential evolution algorithm with multi-mutation strategy," in *2019 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2019, pp. 1236–1242.
[44] P. B. Dao, "On Wilcoxon rank sum test for condition monitoring and fault detection of wind turbines," *Applied Energy*, vol. 318, p. 119209, 2022.
[45] J. Carrasco, S. García, M. Rueda, S. Das, and F. Herrera, "Recent trends in the use of statistical tests for comparing swarm and evolutionary computing algorithms: Practical guidelines and a critical review," *Swarm and Evolutionary Computation*, vol. 54, p. 100665, 2020.
[46] A. Gissler, "Evaluation of the impact of various modifications to cma-es that facilitate its theoretical analysis," in *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, 2023, pp. 1603–1610.
[47] R. Tanabe and A. Fukunaga, "Success-history based parameter adaptation for differential evolution," in *2013 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2013, pp. 71–78.
[48] P. Bujok and P. Kolenovsky, "Eigen Crossover in Cooperative Model of Evolutionary Algorithms Applied to CEC 2022 Single Objective Numerical Optimisation," in *2022 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2022, pp. 1–8.
[49] Ó. Espinoza, K. Rodríguez-Vázquez, C. I. Hernández, and S. Rodriguez-Romo, "Comparison Of Three Versions Of Whale Optimization Algorithm (WOA) On The Bbob Test Suite," in *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, 2023, pp. 1595–1602.
[50] J.-Y. Li, K.-J. Du, Z.-H. Zhan, H. Wang, and J. Zhang, "Distributed Differential Evolution With Adaptive Resource Allocation," *IEEE Transactions on Cybernetics*, vol. 53, no. 5, pp. 2791–2804, 2023.
[51] X. Xia, L. Gui, F. Yu, H. Wu, B. Wei, Y.-L. Zhang, and Z.-H. Zhan, "Triple Archives Particle Swarm Optimization," *IEEE Transactions on Cybernetics*, vol. 50, no. 12, pp. 4862–4875, 2020.
[52] B. Morales-Castañeda, D. Zaldivar, E. Cuevas, F. Fausto, and A. Rodríguez, "A better balance in metaheuristic algorithms: Does it exist?" *Swarm and Evolutionary Computation*, vol. 54, p. 100671, 2020.

Yaotong Song received the B.S. degree from Beijing University of Civil Engineering and Architecture, Beijing, China. He is currently pursuing the M.E. degree from the University of Toyama, Toyama, Japan. His current interests include computational intelligence and neural networks for real-world applications.

Kaiyu Wang received the B.S. degree from Southwest University, Chongqing, China, in 2018, and the M.E. degree from the University of Toyama, Toyama, Japan, in 2022, where he is currently pursuing the Ph.D. degree. His current interests include computational intelligence and neural networks for real-world applications.

Zhi-Hui Zhan (Fellow, IEEE) received the Ph.D. degree in computer science from Sun Yat-Sen University, Guangzhou China, in 2013. He is currently a Changjiang Scholar Young Professor and Gifted Professor with the College of Artificial Intelligence, Nankai University, Tianjin, China. His current research interests include evolutionary computation, swarm intelligence, and their applications in real-world problems. He is currently an Associate Editor of IEEE Transactions on Evolutionary Computation.

Zhenyu Lei received the Ph.D. degree in Science and Engineering from the University of Toyama, Toyama, Japan, in 2023. He is currently an Assistant Professor with the Faculty of Engineering, University of Toyama, Japan. His current research interests include evolutionary computation, machine learning, and neural network for real-world applications and optimization problems.

Shangce Gao (Senior Member, IEEE) received his Ph.D. degree in Innovative Life Science from University of Toyama, Toyama, Japan in 2011. He is currently a Professor with the Faculty of Engineering, University of Toyama, Japan. His current research interests include nature-inspired technologies, machine learning, and neural networks for real-world applications. He serves as an Associate Editor for many international journals such as IEEE Transactions on Neural Networks and Learning Systems, and IEEE/CAA Journal of Automatica Sinica.